

Processing Sequential Files

Run of SeqWrite

Enter student details using template below

```
NNNNNNNNSSSSSSSSIIYYMMDDCCCCGGGGS
9456789COUGHLANMS580812LM510598M
NNNNNNNNSSSSSSSSIIYYMMDDCCCCGGGGS
9367892RYAN      TG521210LM601222F
NNNNNNNNSSSSSSSSIIYYMMDDCCCCGGGGS
9368934WILSON   HR520323LM610786M
NNNNNNNNSSSSSSSSIIYYMMDDCCCCGGGGS
CarriageReturn
```

PROCEDURE DIVISION.

Begin.

OPEN OUTPUT StudentFile

DISPLAY "Enter student details using template below. Press CR to end.".

PERFORM GetStudentDetails

PERFORM UNTIL StudentDetails = SPACES

WRITE StudentDetails

PERFORM GetStudentDetails

END-PERFORM

CLOSE StudentFile

STOP RUN.

GetStudentDetails.

DISPLAY "NNNNNNNNSSSSSSSSIIYYMMDDCCCCGGGGS".

ACCEPT StudentDetails.

```
$ SET SOURCEFORMAT"FREE"
IDENTIFICATION DIVISION.
PROGRAM-ID. SeqWrite.
AUTHOR. Michael Coughlan.
```

ENVIRONMENT DIVISION.

INPUT-OUTPUT SECTION.

FILE-CONTROL.

```
SELECT StudentFile ASSIGN TO "STUDENTS.DAT"
        ORGANIZATION IS LINE SEQUENTIAL.
```

DATA DIVISION.

FILE SECTION.

FD StudentFile.

01 StudentDetails.

02 StudentId PIC 9(7).

02 StudentName.

03 Surname PIC X(8).

03 Initials PIC XX.

02 DateOfBirth.

03 YOBirth PIC 9(2).

03 MOBirth PIC 9(2).

03 DOBirth PIC 9(2).

02 CourseCode PIC X(4).

02 Grant PIC 9(4).

02 Gender PIC X.

RUN OF SeqRead

9456789 COUGHLANMS LM51

9367892 RYAN TG LM60

9368934 WILSON HR LM61

PROCEDURE DIVISION.

Begin.

OPEN INPUT StudentFile

READ StudentFile

AT END MOVE HIGH-VALUES TO StudentDetails

END-READ

PERFORM UNTIL StudentDetails = HIGH-VALUES

DISPLAY StudentId SPACE StudentName SPACE CourseCode

READ StudentFile

AT END MOVE HIGH-VALUES TO StudentDetails

END-READ

END-PERFORM

CLOSE StudentFile

STOP RUN.

\$ SET SOURCEFORMAT"FREE"
IDENTIFICATION DIVISION.
PROGRAM-ID. SeqRead.
AUTHOR. Michael Coughlan.

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.

SELECT StudentFile ASSIGN TO "STUDENTS.DAT"
ORGANIZATION IS LINE SEQUENTIAL.

DATA DIVISION.
FILE SECTION.
FD StudentFile.
01 StudentDetails.
02 StudentId PIC 9(7).
02 StudentName.
03 Surname PIC X(8).
03 Initials PIC XX.
02 DateOfBirth.
03 YOBirth PIC 9(2).
03 MOBirth PIC 9(2).
03 DOBirth PIC 9(2).
02 CourseCode PIC X(4).
02 Grant PIC 9(4).
02 Gender PIC X.

Organization and Access

- ◆ Two important characteristics of files are
 - DATA ORGANIZATION
 - METHOD OF ACCESS
- ◆ Data organization refers to the way the records of the file are organized on the backing storage device. COBOL recognizes three main file organizations;
 - Sequential - Records organized serially.
 - Relative - Relative record number based organization.
 - Indexed - Index based organization.
- ◆ The method of access refers to the way in which records are accessed.
 - A file with an organization of Indexed or Relative may still have its records accessed sequentially.
 - But records in a file with an organization of Sequential can not be accessed directly.



Sequential Organization

- ◆ The simplest COBOL file organization is **Sequential**.
- ◆ In a Sequential file the records are arranged **serially**, one after another, like cards in a dealing shoe.
- ◆ In a Sequential file the only way to access any particular record is to;
Start at the first record and read all the succeeding records until you find the one you want or reach the end of the file.
- ◆ Sequential files may be
Ordered
or
Unordered (these should be called Serial files)
- ◆ The ordering of the records in a file has a significant impact on the way in which it is processed and the processing that can be done on it.



Ordered and Unordered Files

Ordered File

RecordA
RecordB
RecordG
RecordH
RecordK
RecordM
RecordN

Unordered File

RecordM
RecordH
RecordB
RecordN
RecordA
RecordK
RecordG

In an ordered file the records are sequenced on some field in the record.



Adding records to unordered files

Transaction File

RecordF

RecordP

RecordW

PROGRAM

FILE SECTION.

TFRec

UFRec

PROCEDURE DIVISION.

OPEN EXTEND UF.

OPEN INPUT TF.

READ TF.

MOVE TFRec TO UFRec.

WRITE UFRec.

Unordered File

RecordM

RecordH

RecordB

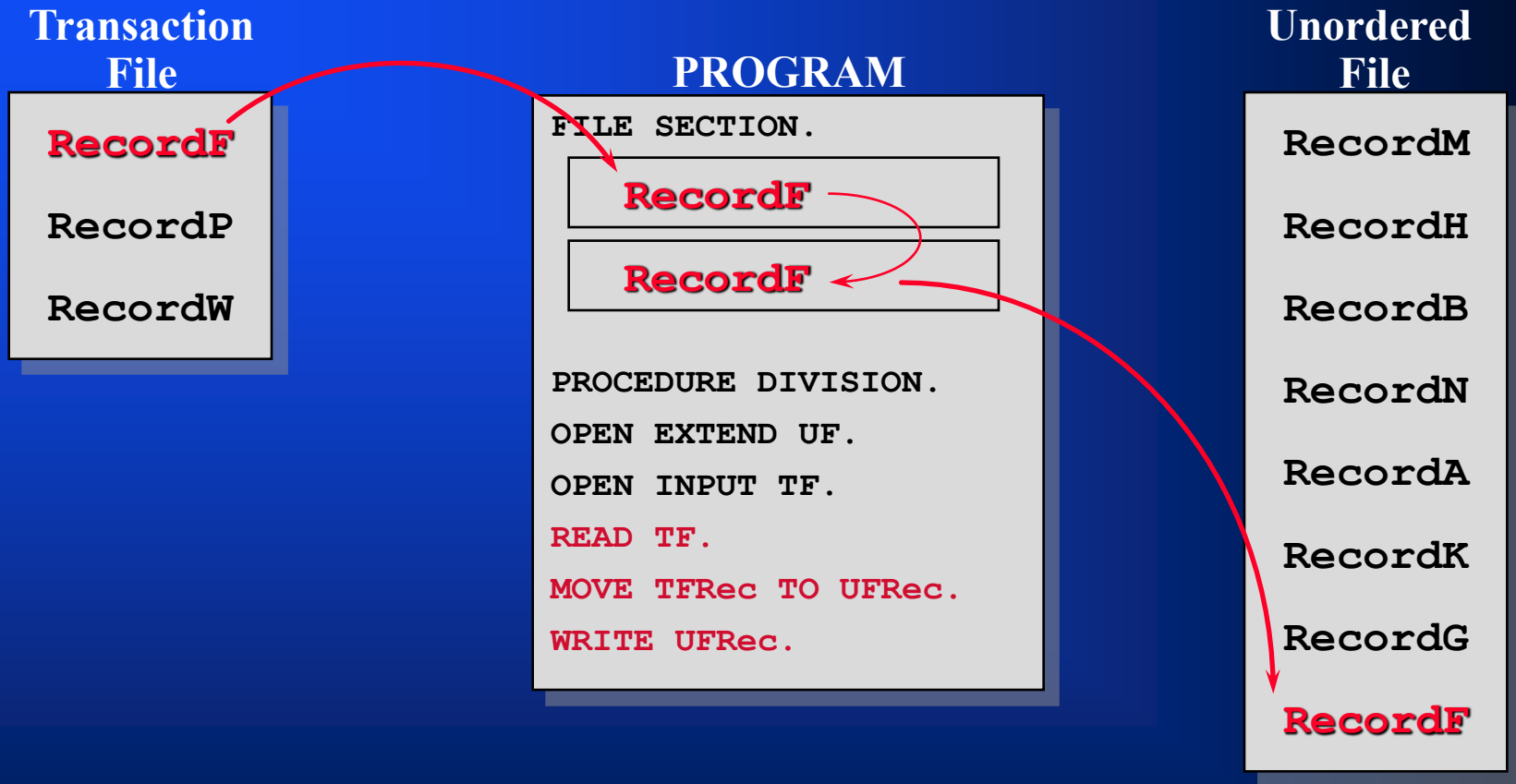
RecordN

RecordA

RecordK

RecordG

Adding records to unordered files



Adding records to unordered files

Transaction
File

RecordF

RecordP

RecordW

Unordered
File

RecordM

RecordH

RecordB

RecordN

RecordA

RecordK

RecordG

RecordF

RecordP

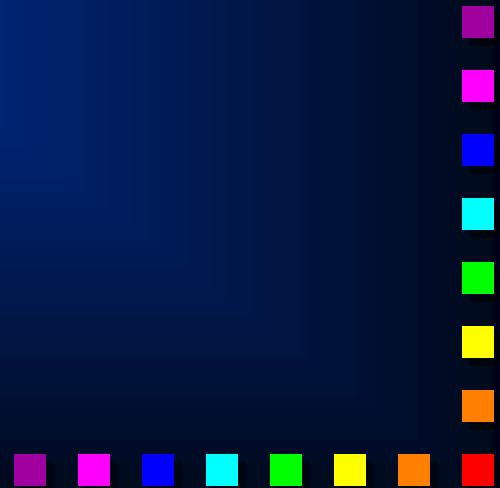
RecordW

RESULT

```
graph LR; TF[Transaction File] --> R[RESULT]; R --> UF[Unordered File]
```

Problems with Unordered Sequential Files

- ◆ It is easy to **add** records to an unordered Sequential file.
- ◆ But it is not really possible to **delete** records from an unordered Sequential file.
- ◆ And it is not feasible to **update** records in an unordered Sequential file



Problems with Unordered Sequential Files

- ◆ Records in a Sequential file can not be deleted or updated “in situ”.
- ◆ The only way to delete Sequential file records is to create a new file which does not contain them.
- ◆ The only way to update records in a Sequential File is to create a new file which contains the updated records.
- ◆ Because both these operations rely on record matching they do not work for unordered Sequential files.
- ◆ Why?



Deleting records from unordered files?

Transaction File



NO

New File



Unordered File



Deleting records from unordered files?

Transaction File



NO

New File



Unordered File



Deleting records from unordered files?

Transaction File

RecordB

RecordM

RecordK

Unordered File

RecordM

RecordH

RecordB

RecordN

RecordA

RecordK

Delete UF
Record?

YES

New File

RecordM

RecordH

Deleting records from unordered files?

Transaction File

RecordB

RecordM

RecordK

Unordered File

RecordM

RecordH

RecordB

RecordN

RecordA

RecordK

Delete UF
Record?

NO

New File

RecordM

RecordH

RecordN

But wait...

We should have deleted RecordM. Too late. It's already been written to the new file.

Deleting records from an ordered file

Transaction File

RecordB

RecordK

RecordM

Ordered File

RecordA

RecordB

RecordG

RecordH

RecordK

RecordM

RecordN

PROGRAM

FILE SECTION.

TFRec

OFRec

NFRec

PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF TFKey NOT = OFKey

MOVE OFRec TO NFRec

WRITE NFRec

READ OF

ELSE

READ TF

READ OF

END-IF.

New File

Deleting records from an ordered file

Transaction File

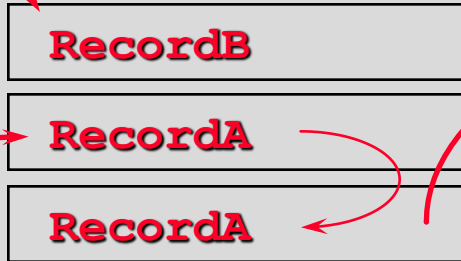


Ordered File



PROGRAM

FILE SECTION.



PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF TFRec NOT = OFRec

MOVE OFRec TO NFRec

WRITE NFRec

READ OF

ELSE

READ TF

READ OF

END-IF.

New File



Problem !!
How can we
recognize which
record we want to
delete?

By its Key Field

Deleting records from an ordered file

Transaction File

RecordB
RecordK
RecordM

Ordered File

RecordA
RecordB
RecordG
RecordH
RecordK
RecordM
RecordN

PROGRAM

FILE SECTION.

RecordB

RecordB

RecordA

PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF TFKey NOT = OFKey

MOVE OFRec TO NFRec

WRITE NFRec

READ OF

ELSE

READ TF

READ OF

END-IF.

New File

RecordA

Deleting records from an ordered file

Transaction File



Ordered File



PROGRAM

FILE SECTION.



PROCEDURE DIVISION.

```
OPEN INPUT TF.  
OPEN INPUT OF  
OPEN OUTPUT NF.  
READ TF.  
READ OF.  
IF TFKey NOT = OFKey  
    MOVE OFRec TO NFRec  
    WRITE NFRec  
    READ OF  
ELSE  
    READ TF  
    READ OF  
END-IF.
```

New File



Deleting records from an ordered file

Transaction File

RecordB

RecordK

RecordM

Ordered File

RecordA

RecordB

RecordG

RecordH

RecordK

RecordM

RecordN

RESULT

New File

RecordA

RecordG

RecordH

RecordN



Updating records in an ordered file

Transaction File

RecordB

RecordH

RecordK

Ordered File

RecordA

RecordB

RecordG

RecordH

RecordK

RecordM

RecordN

PROGRAM

FILE SECTION.

TFRec

OFRec

NFRec

PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF TFKey = OFKey

Update OFRec with TFRec

MOVE OFRec+ TO NFRec

WRITE NFRec

READ TF

READ OF

ELSE

MOVE OFRec TO NFRec

WRITE NFRec

READ OF

END-IF.

New File

Updating records in an ordered file

Transaction File

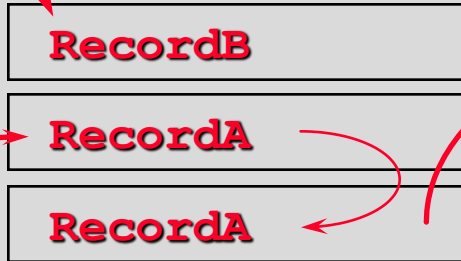


Ordered File



PROGRAM

FILE SECTION.



PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF TFKey = OFKey

Update OFRec with TFRec

MOVE OFRec+ TO NFRec

WRITE NFRec

READ TF

READ OF

ELSE

MOVE OFRec TO NFRec

WRITE NFRec

READ OF

END-IF.

New File



Updating records in an ordered file

Transaction File

RecordB
RecordH
RecordK

Ordered File

RecordA
RecordB
RecordG
RecordH
RecordK
RecordM
RecordN

PROGRAM

FILE SECTION.

RecordB

RecordB

RecordB+

PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF **TFKey** = **OFKey**

Update OFRec with TFRec

MOVE OFRec+ TO NFRec

WRITE NFRec

READ TF

READ OF

ELSE

MOVE OFRec TO NFRec

WRITE NFRec

READ OF

END-IF.

New File

RecordA
RecordB+

Updating records in an ordered file

Transaction File

RecordB
RecordH
RecordK

Ordered File

RecordA
RecordB
RecordG
RecordH
RecordK
RecordM
RecordN

PROGRAM

FILE SECTION.

RecordH
RecordG
RecordG

PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF TFKey = OFKey

Update OFRec with TFRec

MOVE OFRec+ TO NFRec

WRITE NFRec

READ TF

READ OF

ELSE

MOVE OFRec TO NFRec

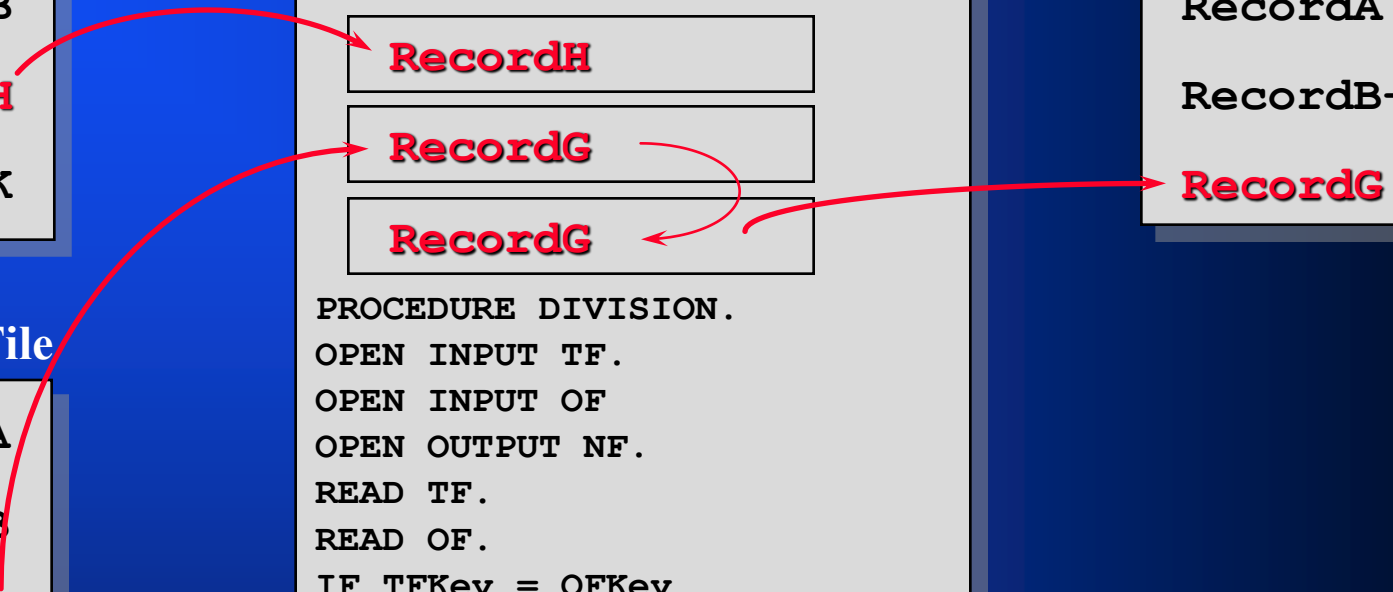
WRITE NFRec

READ OF

END-IF.

New File

RecordA
RecordB+
RecordG



Inserting records into an ordered file

Transaction File

RecordC

RecordF

RecordP

Ordered File

RecordA

RecordB

RecordG

RecordH

RecordK

RecordM

RecordN

PROGRAM

FILE SECTION.

TFRec

OFRec

NFRec

PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF TFKey < OFKey

MOVE TFRec TO NFRec

WRITE NFRec

READ TF

ELSE

MOVE OFRec TO NFRec

WRITE NFRec

READ OF

END-IF.

New File

Inserting records into an ordered file

Transaction File

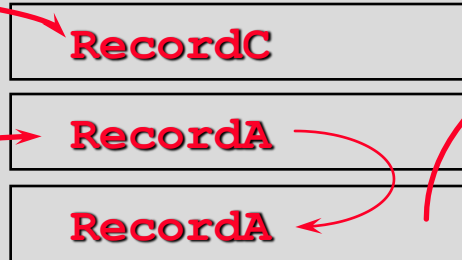


Ordered File



PROGRAM

FILE SECTION.



PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF TFKey < OFKey

MOVE TFRec TO NFRec

WRITE NFRec

READ TF

ELSE

MOVE OFRec TO NFRec

WRITE NFRec

READ OF

END-IF.

New File



Inserting records into an ordered file

Transaction File

RecordC
RecordF
RecordP

Ordered File

RecordA
RecordB
RecordG
RecordH
RecordK
RecordM
RecordN

PROGRAM

FILE SECTION.

RecordC

RecordB

RecordB

PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF TFKey < OFKey

MOVE TFRec TO NRec

WRITE NRec

READ TF

ELSE

MOVE OFRec TO NRec

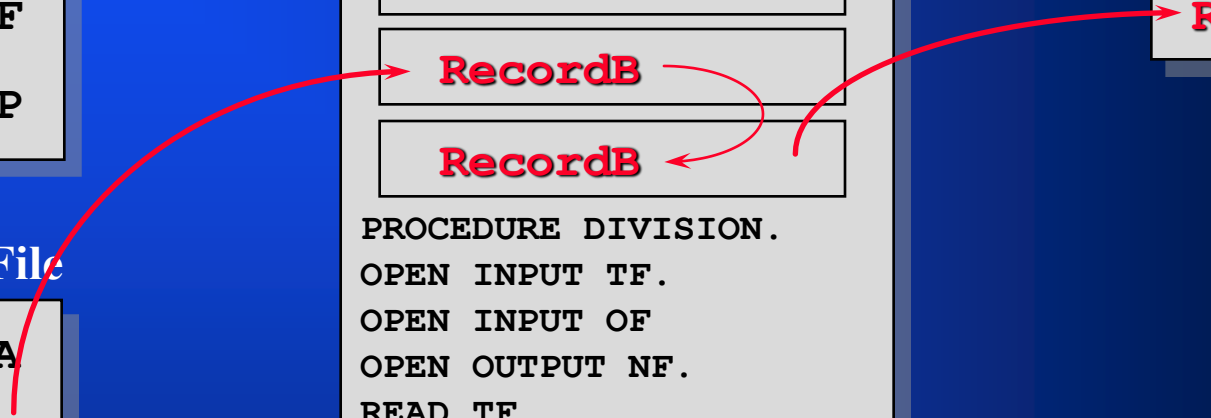
WRITE NRec

READ OF

END-IF.

New File

RecordA
RecordB



Inserting records into an ordered file

Transaction File

RecordC
RecordF
RecordP

PROGRAM

FILE SECTION.

RecordC

RecordG

RecordC

PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF TFKey < OFKey

MOVE TFRec TO NFRec

WRITE NFRec

READ TF

ELSE

MOVE OFRec TO NFRec

WRITE NFRec

READ OF

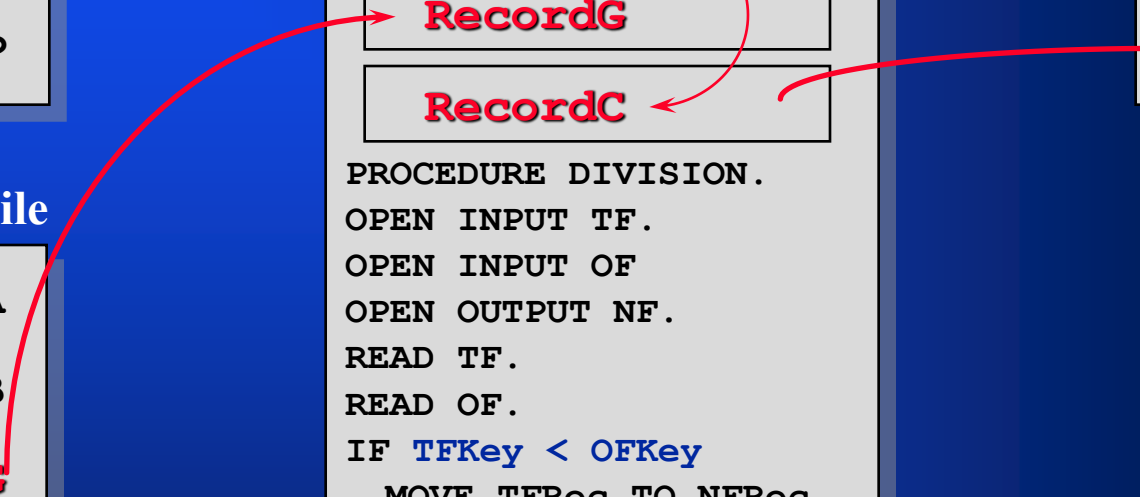
END-IF.

New File

RecordA
RecordB
RecordC

Ordered File

RecordA
RecordB
RecordG
RecordH
RecordK
RecordM
RecordN



Inserting records into an ordered file

Transaction File

RecordC

RecordF

RecordP

Ordered File

RecordA

RecordB

RecordG

RecordH

RecordK

RecordM

RecordN

PROGRAM

FILE SECTION.

RecordF

RecordG

RecordF

PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF **TFKey** < **OFKey**

MOVE TFRec TO NFRec

WRITE NFRec

READ TF

ELSE

MOVE OFRec TO NFRec

WRITE NFRec

READ OF

END-IF.

New File

RecordA

RecordB

RecordC

RecordF

Inserting records into an ordered file

Transaction File

RecordC

RecordF

RecordP

Ordered File

RecordA

RecordB

RecordG

RecordH

RecordK

RecordM

RecordN

PROGRAM

FILE SECTION.

RecordP

RecordG

RecordG

PROCEDURE DIVISION.

OPEN INPUT TF.

OPEN INPUT OF

OPEN OUTPUT NF.

READ TF.

READ OF.

IF TFKey < OFKey

MOVE TFRec TO NFRec

WRITE NFRec

READ TF

ELSE

MOVE OFRec TO NFRec

WRITE NFRec

READ OF

END-IF.

New File

RecordA

RecordB

RecordC

RecordF

RecordG